Individual Reflection

41052 Advanced Algorithms - Programming Assignment 2

Bin Packing: Integer Linear Program (Exact)

November 4, 2026

Swas Dwivedi
25541691

Worst case and the inputs that trigger it

ILP uses the formulation: a binary y_j for each bin slot and a binary x_{ij} for each item and bin, solved over HiGHS under a 10s limit (FFD's bin count is passed as U, the number of slots). Its worst case is not about finding a good packing. It is about proving that no better packing with fewer bins exists. That's the harddest proof when the optimum sits exactly one bin above the lower bound, because the solver then has to rule out every LB-bin packing by search.

The ANI set is built to mimic exactly this and it highlights the gap perfectly in our results. FFD reached the optimum on all 250 instances, almost instantly. ILP on the other hand, returned the optimum on only 81 of them. On rest it reached the time limit with no feasible solution at all, even though a feasible packing with U bins was guaranteed to exist. The AI and SS triples sets behaved the same way: the ILP never proved optimality on a single instance, they all reached the time limit.

The second trigger is plain size. The model has $n*U$ variables. The first GI instance has $n = 1227$ and $U = 532$, so about 650,000 binaries before the solver does anything. HiGHS found no feasible solution on any of the 240 GI instances. For comparison, the ILP was comfortable on Scholl_1 and Scholl_2, with a median of 642ms and 394ms.

bug and design decision

The bug that costs us the most time in inplemnentation was a kinda silly, I first stored the constraint indices as `std::size_t`, because that is what I normally use for any indices, and that broke the HiGHS calls. HiGHS takes index arrays as `HighsInt` (a 32-bit signed int in a default build). An unsigned 64-bit `size_t` array being of different type, could not be passed to the row-building calls.

Additionally, I found another bug after analysing the results. On a few instances like `Scholl_1 and N3C1W1_L` with $n = 200$, $C = 100$) my L2 lower bound already equalled FFD's bin count ($97 = 97$), so FFD's packing was provably optimal before HiGHS even started. My ILP still ran the full 10 s and returned -1. When LB $= U$, my code fixes every $y_j = 1$, so the objective becomes a constant and HiGHS has to find a packing into exactly 97 bins from scratch, with no starting solution, since I return -1 for both "timed out" and "infeasible", and our analysis script drops -1 rows, this simply looked like the ILP being slow.

The design decision was with symmetry. Bins are interchangeable, so any packing appears up to $U!$ times in the search tree, and branch and bound wastes most of its time revisiting the same solution, for which I added two restrictions. First, $y_j \geq y_{j+1}$, so the used bins always come first. Second, item i may only go in bins $0, \ldots, i$. With the items sorted in decreasing order, the largest items are pinned to the first few bins, which removes most relabelled copies without cutting off any optimal packing.

One thing I now understand

I assumed an exact solver is slow because it searches through many packings. What I understood later, is that most of its time goes into closing the gap between the best packing found and the lower bound also, the LP relaxation of the model is weak: fractional items can split across bins, so its bound is essentially $\sum w_i/C$. That is why I added Lower bound 2(L2). Fixing the first L2 bins as used tightens the floor the solver has to prove, and since the objective is an integer, a gap below 1 already proves optimality. I didn't benchmark either change in isolation, so I can't say how much each helped. But the ANI results support the underlying point i.eii on the 81 instances where the ILP returned the optimal count, it still ran the full 10 s. It had found the answer but couldn't prove it.

What I would do differently

First, I would pass FFD's packing to HiGHS as a starting solution setSolution. A U-bin packing always exists, so the ILP could never return nothing. This would also cover the 169 ANI instances that

failed even though LB $< U$. I would also return immediately when LB $= U$, and give timeouts their own status instead of reusing -1.

Second, I would try a pattern-based (set partitioning) formulation instead of item-to-bin assignment as it removes bin symmetry entirely, so my two symmetry constraints would not be needed, and its LP bound matches the optimum on almost all practical instances. ANI is built so that even this bound is one bin short, so it would be a real test of whether a stronger bound actually helps there.